

Lecture2Quiz SEA

From classroom audio to bilingual study packs

Valsea Speech AI

AWS Bedrock

FastAPI

React + Tailwind

Press → or Space to navigate | Esc for overview | ?
for help

The Problem

- Students in Southeast Asia attend lectures in **mixed languages** (English + local)
- Taking notes manually is **slow and incomplete**
- No easy way to generate **quizzes or flashcards** from a lecture
- Existing tools don't understand **SEA accents & languages** (Singlish, Vietnamese, Thai, etc.)

Our Solution

Upload a lecture recording → Get a complete **bilingual study pack**:

Outputs

- Clean transcript
- Key quotes & overview
- Action items / takeaways
- Bilingual glossary
- 10-question quiz
- Leveled flashcards (easy / medium / hard)

Key Features

- Supports **70+ languages**
- Auto-splits large audio
- Parallel processing
- Real-time SSE progress
- Spaced-repetition study
- Quiz miss → flashcard

Architecture Overview

Browser (React + Tailwind)



FastAPI Backend (Python)



Valsea API

transcribe, clarify, format, translate

AWS Bedrock (Claude)

quiz generation, flashcards

Processing Pipeline

1. Upload Audio



2. Auto-Split

(if > 8 MB)



3. Transcribe

Valsea STT



4. Clarify

clean up noisy text



5. Format + Quiz

parallel (Valsea + Bedrock)



6. Translate

→ target language

Steps 5 runs formatting (key quotes, overview, takeaways), quiz gen, and flashcard gen **all in parallel**.

Valsea APIs Used

Endpoint	Model	Purpose
POST /v1/audio/transcriptions	valsea-transcribe	Speech-to-text with SEA accent support
POST /v1/clarifications	valsea-clarify	Clean noisy/colloquial transcript
POST /v1/formatting	valsea-format	Generate key_quotes, meeting_minutes, action_items
POST /v1/translations	valsea-translate	Translate to student's native language

All powered by **valsea.ai** — built specifically for Southeast Asian languages.

AWS Bedrock (Claude)

Used for **intelligent content generation** from the clean transcript:

Quiz Generation

- 10 multiple-choice questions
- 4 options each (A–D)
- Correct answer + explanation
- Covers key concepts from lecture

Flashcard Generation

- Front (question) / Back (answer)
- 3 difficulty levels: easy, medium, hard
- Card types: definition, concept, application
- Feeds into spaced-repetition engine

Frontend Experience

Page	Route	What it does
Home	/	Upload audio, select languages, see real-time progress (SSE)
Library	/library	Browse past sessions, view bilingual summaries & glossary
Quiz	/quiz	Interactive quiz room — immediate feedback, score tracking
Flashcards	/flashcards	Flip cards with spaced-repetition, personalized from quiz misses

Smart Audio Handling

Large lecture recordings (> 8 MB) are handled automatically:

1. Detect file exceeds threshold → trigger split
2. Split into ~4.5-minute MP3 chunks via `ffmpeg`
3. Transcribe all chunks **in parallel** (3 concurrent by default)
4. Recombine text + semantic tags in order
5. Continue pipeline with full combined transcript

No manual splitting needed — just upload the full recording (up to 1 hour).

Real-Time Progress (SSE)

The UI shows live pipeline status via [Server-Sent Events](#):

```
POST /process?stream=true
```

```
← {"type":"phase","phase":"transcribe","label":"Transcribing audio..."}  
← {"type":"phase","phase":"clarify","label":"Clarifying transcript..."}  
← {"type":"phase","phase":"summary_quiz","label":"Generating study pack..."}  
← {"type":"phase","phase":"translate","label":"Translating → vietnamese..."}  
← {"type":"complete","payload":{"transcript", "summary", "quiz", "flashcards"}}
```

Students see exactly what's happening — no mystery loading spinner.

Student Study Flow

Upload Lecture



Read Summary
(bilingual)



Take Quiz



Review Flashcards
(includes quiz misses!)



Spaced Repetition
(localStorage)

Wrong quiz answers automatically become flashcards for extra review.

Tech Stack

Backend

- **Python** + FastAPI
- httpx (async HTTP)
- boto3 (AWS Bedrock)
- ffmpeg (audio splitting)
- pytest + respx (testing)

Frontend

- **React** + React Router
- Vite (build tool)
- Tailwind CSS
- localStorage (persistence)
- EventSource (SSE)

API Design

Method	Path	Description
GET	/health	Liveness check
POST	/process	Main pipeline endpoint multipart: <code>audio</code> , <code>target_language</code> , <code>transcription_language</code> , <code>stream</code> Returns: transcript, summary (bilingual), quiz, flashcards

Single endpoint does everything — upload once, get the complete study pack.

Quick Demo (Mock Mode)

Run without API keys using mock data:

```
# .env  
USE MOCK QUIZ=true  
USE MOCK FLASHCARDS=true
```

This loads pre-built JSON fixtures so you can demo the UI without:

- Valsea API key
- AWS credentials
- Network connectivity

Language Support

Valsea supports **70+ languages** with accent-aware models:

English

Vietnamese

Thai

Indonesian

Malay

Filipino

Chinese

Japanese

Korean

Singlish

Hindi

Khmer

Lao

Javanese

Tamil

Bengali

+ 55 more...

Including regional variants: English-PH, Arabic-UAE, Bengali-BD, etc.

Running the Project

```
# Terminal 1 – Backend  
cd backend  
python3 -m venv .venv && source .venv/bin/activate  
pip install -r requirements.txt  
uvicorn main:app --reload --port 8001
```

```
# Terminal 2 – Frontend  
cd frontend  
npm install && npm run dev
```

Open <http://127.0.0.1:5173> → Upload audio → Watch the magic happen.

What Makes This Unique

1. **SEA-first** — built on Valsea, which understands Singlish, accented English, and local languages
2. **End-to-end** — one upload produces transcript + summary + quiz + flashcards
3. **Bilingual output** — everything in English + student's native language
4. **Adaptive learning** — quiz mistakes feed into flashcard deck automatically
5. **No manual work** — auto-split, parallel processing, smart retries

Roadmap

- Real-time live transcription (WebSocket via Valsea RTT)
- Persistent session database (SQLite → PostgreSQL)
- Analytics dashboard (study progress, weak topics)
- Collaborative study rooms
- Mobile-friendly PWA
- Export to PDF / Anki deck

Thank You

Lecture2Quiz SEA

From classroom audio to bilingual study packs

GitHub: [VALSEA_PROJECT](#)

API: [valsea.ai](#)

Print to PDF: append **?print-pdf** to the URL, then
Ctrl+P / Cmd+P

Speaker notes